

Generic Collections in C# with Examples

In this article, I am going to give a brief introduction to **Generic Collections in C# with Examples**. Please read our previous article where we discussed the [Advantages and Disadvantages of the Non-Generic Collection in C#](#) with Examples. As part of this article, first, we will discuss the problems of Non-Generic Collections and then we will discuss how to overcome the Non-Generic Collection problems with Generic Collections in C#. Finally, we will discuss the generic collection class examples in C#.

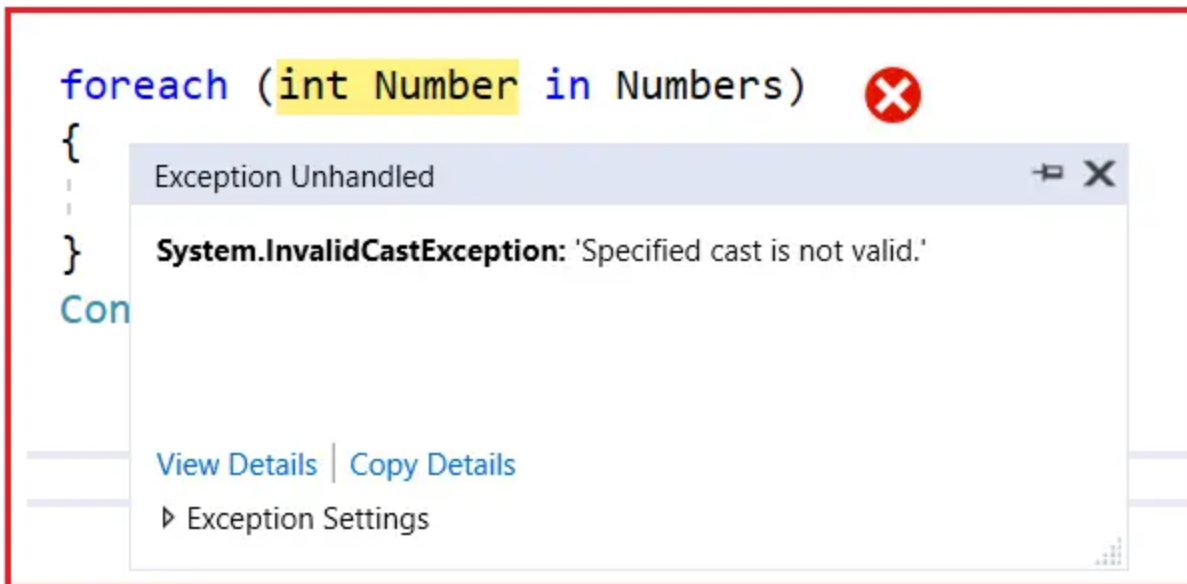
Note: Generic Collections are introduced as part of **C# 2.0**. You can consider this Generic Collection as an extension to the Non-Generic Collection Classes which we have already discussed in our previous articles such as [ArrayList](#), [Hashtable](#), [SortedList](#), [Stack](#), and [Queue](#).

Problems with Non-Generic Collections in C#

The Non-Generic Collection Classes such as **ArrayList**, **Hashtable**, **SortedList**, **Stack**, and **Queue** are worked on the object data type. That means the elements added to the collection are of an object type. As these Non-Generic CollectionClasses worked on object data type, we can store any type of value that may lead to runtime exceptions due to type mismatch. But with **Generic Collections**, now we are able to store a specific type of data (whether a primitive type or a reference type) which provides type safety by eliminating the type mismatch at run time. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace CollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ArrayList Numbers = new ArrayList(3);
            Numbers.Add(100);
            Numbers.Add(200);
            Numbers.Add(300);
            //It is also possible to store string values
            Numbers.Add("Hi");
            foreach (int Number in Numbers)
            {
                Console.Write(Number + " ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:



The Second Problem with Non-Generic Collection Classes is that we get performance overhead. The reason for this is Boxing and Unboxing. As we already discussed these collection classes worked on the object data type. So if we are storing value-type data in the collection, then those value-type data are first converted into object type and then only store into the collection which is nothing but performing Boxing. Similarly, if we want to retrieve the data from the collection, then we need to convert the data from object type to value type means performing Unboxing. Due to this Boxing and Unboxing, we get poor performance if our collection is a big one. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections;
namespace CollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ArrayList Numbers = new ArrayList(2);
            // Boxing happens - Converting Value type (100, 200) to reference
            // Means Integer to object type
            Numbers.Add(100);
            Numbers.Add(200);
            // Unboxing happens - 100 and 200 stored as object type
            // now converted to Integer type
            foreach (int Number in Numbers)
            {
                Console.Write(Number + " ");
            }
            Console.ReadKey();
        }
    }
}
```

Note: Boxing means converting a value type to an object type and Unboxing means converting an object type back to the value type.

The Solution to Non-Generic Collection Problems

The above two problems of Non-Generic Collections are overcome by using the Generic Collections in C#. The .NET Framework has re-implemented all the existing Non-Generic Collection classes such as **ArrayList**, **Hashtable**, **SortedList**, **Stack**, and **Queue**., etc. in Generic Collections such as **ArrayList<T>**, **Dictionary<TKey, TValue>**, **SortedList<TKey, TValue>**, **Stack<T>**, and **Queue<T>**. Here T is nothing but the type of values that we want to store in the collection. So, the most important point that you need to remember is while creating the objects of Generic Collection Classes, you need to explicitly provide the type of values that you are going to store in the collection.

A Generic Collection is Strongly Type-Safe. Which type of data do you want to store in generic type, this information you have to provide at compile time. It means you can only put one type of data into it. This eliminates type mismatches at runtime.

The Generic Collection Classes are implemented under the **System.Collections.Generic** namespace. The classes which are present in this namespace are as follows.

1. **Stack<T>**: It represents a variable size last-in-first-out (LIFO) collection of instances of the same specified type.
2. **Queue<T>**: It represents a first-in, first-out collection of objects.
3. **HashSet<T>**: It represents a set of values. It eliminates duplicate elements.
4. **SortedList<TKey, TValue>**: It represents a collection of key/value pairs that are sorted by key based on the associated **System.Collections.Generic.IComparer** implementation. It automatically adds the elements in ascending order of key by default.
5. **List<T>**: It represents a strongly typed list of objects that can be accessed by index. Provides methods to search, sort, and manipulate lists. It grows automatically as you add elements to it.
6. **Dictionary<TKey, TValue>**: It represents a collection of keys and values.
7. **SortedSet<T>**: It represents a collection of objects that are maintained in sorted order.
8. **SortedDictionary<TKey, TValue>**: It represents a collection of key/value pairs that are sorted on the key.
9. **LinkedList<T>**: It represents a doubly linked list.

Note: Here the <T> refers to the type of values we want to store under them.

Examples:

```
List<int> listOfInteger = new List<int>();    // stores integer values only
List<float> listOfFloat = new List<float>(); // stores float values only
List<string> listOfString = new List<string>(); // stores string values only
```

It is also possible to store a user-defined type like a class type or structure type as shown below

```
List<Customer> customerList = new List<Customer> ();
```

Assume the **Customer** is a user-defined class type that represents an entity Customer, Here we can store the customer objects within the customerList collection where each customer object can internally represent different attributes of the customer like Id, Name, City, State, etc.

What are Generic Collections in C#?

The Generic Collections in C# are strongly typed. The strongly typed nature allows these collection classes to store only one type of value into it. This not only eliminates the type mismatch at runtime but also will get better performance as they don't require boxing and unboxing while they store value type data. So, it is always a preferable and a good programming choice to use the Generics Collection Classes in C# rather than using the Non-Generic Collection Classes.

Note: In most cases, it is recommended to use the generic collections because they perform faster than non-generic collections and also minimize exceptions by giving compile-time errors.

In this article, I am going to give the use of each generic collection class with a simple example and from our next article onwards, we will explain each generic collection class in detail.

List<T> Class in C#

The List<T> Generic Collection Class in C# is used to store and fetch elements. It can have duplicate elements. It belongs to System.Collections.Generic namespace. You can also consider the List<T> collection as the generic version of ArrayList. Here, we need to mention the type of values that we want to store in the collection. Like, ArrayList we are unable to add any types of value into the List<T> collection, which prevents us from runtime exceptions due to type mismatch. Let's see an example of a generic List<T> collection class that stores elements using Add() method and iterates elements using for-each loop.

```
using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericListDemo
    {
        public static void Main(string[] args)
        {
            List<int> integerList = new List<int>();
            integerList.Add(11);
            integerList.Add(22);
            integerList.Add(55);
            integerList.Add(65);
            integerList.Add(10);
            //The following line give you compile time error as the value is :
            //integerList.Add("Hello");
            Console.Write("List of Elements: ");
            foreach (int item in integerList)
            {
                Console.Write($"{item} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
List of Elements: 11 22 55 65 10
```

For more detailed information on Generic List<T> Collection Class in C#, please click on the below URL.

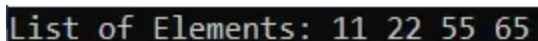
<https://dotnettutorials.net/lesson/list-collection-csharp/>

HashSet<T> Class in C#

The Generic HashSet<T> Collection Class in C# can be used to store, remove or view elements. It does not allow the addition of duplicate elements. It is suggested to use the HashSet class if you have to store only unique elements. It belongs to System.Collections.Generic namespace. Let's see an example of a generic HashSet<T> collection class that stores elements using Add() method and iterates elements using for-each loop.

```
using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericHashSetDemo
    {
        public static void Main(string[] args)
        {
            HashSet<int> integerHashSet = new HashSet<int>();
            integerHashSet.Add(11);
            integerHashSet.Add(22);
            integerHashSet.Add(55);
            integerHashSet.Add(65);
            //Addind Duplicate Elements
            integerHashSet.Add(55);
            //The following line give you compile time error as the value is :
            //integerHashSet.Add("Hello");
            Console.WriteLine("List of Elements: ");
            foreach (int item in integerHashSet)
            {
                Console.WriteLine($"{item} ");
            }
            Console.ReadKey();
        }
    }
}
```

If you notice, we have added 55 elements two times. Now, run the application and you will see that it removes the duplicate element and shows 55 only once as shown in the below image.



List of Elements: 11 22 55 65

For more detailed information on Generic HashSet<T> Collection Class in C#, please click on the below URL.

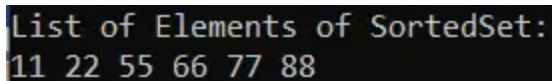
<https://dotnettutorials.net/lesson/generic-hashset-collection-class-in-csharp/>

SortedSet<T> Class in C#:

The Generic SortedSet<T> Collection Class in C# is used to store, remove or view elements. By default, it stores the elements in ascending order and does not store duplicate elements. It is recommended to use if you have to store unique elements as well as if you want to maintain ascending order. The SortedSet<T> class belongs to System.Collections.Generic namespace. Let's see an example of a generic SortedSet<T> collection class in C# that stores elements using Add() method and iterates elements using for-each loop.

```
using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericSortedSetDemo
    {
        public static void Main(string[] args)
        {
            SortedSet<int> integerSortedSet = new SortedSet<int>();
            integerSortedSet.Add(11);
            integerSortedSet.Add(66);
            integerSortedSet.Add(55);
            integerSortedSet.Add(88);
            integerSortedSet.Add(22);
            integerSortedSet.Add(77);
            //Addind Duplicate Elements
            integerSortedSet.Add(55);
            //The following line give you compile time error as the value is :
            //integerSortedSet.Add("Hello");
            Console.WriteLine("List of Elements of SortedSet:");
            foreach (int item in integerSortedSet)
            {
                Console.Write($"{item} ");
            }
            Console.ReadKey();
        }
    }
}
```

As you notice in the above-sorted set, we have added 55 elements two times. Now, run the application and you will see that it removes the duplicate element and shows 55 only once as well as it will sort the elements in ascending order as shown in the below image.



```
List of Elements of SortedSet:
11 22 55 66 77 88
```

For more detailed information on Generic SortedSet<T> Collection Class in C#, please click on the below URL.

<https://dotnettutorials.net/lesson/generic-sortedset-collection-class-in-csharp/>

Stack<T> Class in C#

The Generic Stack<T> Collection Class in C# is used to push and pop elements in LIFO (Last in First Out) order. The push operation adds an element to a collection, whereas the pop operation is used to remove the most recently added element from a collection. It can have duplicate elements. The Stack<T> class belongs to System.Collections.Generic namespace. Let's see an example of a generic Stack<T> collection class in C# that stores elements using the Push() method, removes elements using the Pop() method and iterates elements using for-each loop.

```
using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericStackDemo
    {
        public static void Main(string[] args)
        {
            Stack<string> countriesStack = new Stack<string>();
            countriesStack.Push("India");
            countriesStack.Push("USA");
            countriesStack.Push("UK");
            countriesStack.Push("China");
            countriesStack.Push("Nepal");
            Console.WriteLine("Stack Elements: ");
            foreach (string country in countriesStack)
            {
                Console.Write($"{country} ");
            }
            Console.WriteLine("\n\nPeek Element: " + countriesStack.Peek());
            Console.WriteLine("Element Popped: " + countriesStack.Pop());
            Console.WriteLine("\nStack Elements: ");
            foreach (string country in countriesStack)
            {
                Console.Write($"{country} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
Stack Elements:
Nepal China UK USA India

Peek Element: Nepal
Element Popped: Nepal

Stack Elements:
China UK USA India
```

For more detailed information on Generic Stack<T> Collection Class in C#, please click on the below URL

<https://dotnettutorials.net/lesson/generic-stack-csharp/>

Queue<T> Class in C#:

The Generic Queue<T> Collection Class in C# is used to Enqueue and Dequeue elements in FIFO (First In First Out) order. The Enqueue operation adds an element in a collection, whereas the Dequeue operation is used to remove the firstly added element from the queue collection. It can have duplicate elements. The Queue<T> Collection Class belongs to System.Collections.Generic namespace. Let's see an example of a generic Queue<T> collection class in C# that add elements using Enqueue() method, removes elements using Dequeue() method and iterates elements using the for-each loop.

```
using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericQueueDemo
    {
        public static void Main(string[] args)
        {
            Queue<string> countriesQueue = new Queue<string>();
            countriesQueue.Enqueue("India");
            countriesQueue.Enqueue("USA");
            countriesQueue.Enqueue("UK");
            countriesQueue.Enqueue("China");
            countriesQueue.Enqueue("Nepal");
            Console.WriteLine("Queue Elements: ");
            foreach (string country in countriesQueue)
            {
                Console.Write($"{country} ");
            }
            Console.WriteLine("\n\nPeek Element: " + countriesQueue.Peek());
            Console.WriteLine("Element Removed: " + countriesQueue.Dequeue());
            Console.WriteLine("\nQueue Elements: ");
            foreach (string country in countriesQueue)
            {
                Console.Write($"{country} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```

Queue Elements:
India USA UK China Nepal

s Peek Element: India
Element Removed: India

Queue Elements:
USA UK China Nepal

```

For more detailed information on Generic Queue<T> Collection Class in C#, please click on the below URL

<https://dotnettutorials.net/lesson/generic-queue-collection-class-csharp/>

Dictionary<TKey, TValue> Class in C#:

The Generic Dictionary<TKey, TValue> Collection Class in C# is the generic version of Hashtable. It works as same as the Hashtable except that it operates on a type object, and this is one of the most useful collections based on key and value pairs. It stores values on the basis of keys. It contains unique keys only. With the help of the key, we can easily search or remove elements. The Dictionary<TKey, TValue> Collection Class belongs to System.Collections.Generic namespace. Let's see an example of a generic Dictionary<TKey, TValue> collection class in C# that stores elements using Add() method and iterates elements using for-each loop. Here, we are using KeyValuePair class to get keys and values.

```

using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericDictionaryDemo
    {
        public static void Main(string[] args)
        {
            Dictionary<int, string> dictionary = new Dictionary<int, string>();
            dictionary.Add(1, "One");
            dictionary.Add(2, "Two");
            dictionary.Add(3, "Three");
            dictionary.Add(4, "Four");
            dictionary.Add(5, "Five");
            Console.WriteLine("Dictionary Elements: ");
            foreach (KeyValuePair<int, string> kvp in dictionary)
            {
                Console.WriteLine($"Key: {kvp.Key}, Value: {kvp.Value}");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```
Dictionary Elements:
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three
Key: 4, Value: Four
Key: 5, Value: Five
```

For more detailed information on Generic Dictionary<TKey, TValue> Collection Class in C#, please click on the below URL

<https://dotnettutorials.net/lesson/dictionary-generic-collection-csharp/>

SortedDictionary<TKey, TValue> Class in C#

The Generic SortedDictionary<TKey, TValue> Collection Class in C# works similar to Dictionary<TKey, TValue> collection class. It stores values on the basis of keys. It contains unique keys and the most important thing is it stored the elements in ascending order on the key. With the help of a key, we can easily search or remove elements. The SortedDictionary<TKey, TValue> Collection Class belongs to System.Collections.Generic namespace. Let's see an example of a generic SortedDictionary<TKey, TValue> collection class in C# that stores elements using Add() method and iterates elements using the for-each loop. Here, we are using KeyValuePair class to get keys and values.

```
using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericSortedDictionaryDemo
    {
        public static void Main(string[] args)
        {
            SortedDictionary<int, string> sortedDictionary = new SortedDictionary<
string>();
            sortedDictionary.Add(1, "One");
            sortedDictionary.Add(5, "Five");
            sortedDictionary.Add(2, "Two");
            sortedDictionary.Add(4, "Four");
            sortedDictionary.Add(3, "Three");
            Console.WriteLine("SortedDictionary Elements: ");
            foreach (KeyValuePair<int, string> kvp in sortedDictionary)
            {
                Console.WriteLine($"Key: {kvp.Key}, Value: {kvp.Value}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
SortedDictionary Elements:
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three
Key: 4, Value: Four
Key: 5, Value: Five
```

For more detailed information on Generic SortedDictionary<TKey, TValue> Collection Class in C#, please click on the below URL

<https://dotnettutorials.net/lesson/generic-sorteddictionary-collection-class-in-csharp/>

SortedList<TKey, TValue> Class in C#

The Generic SortedList<TKey, TValue> Collection Class in C# is a collection of key/value pairs that are sorted according to keys. By default, this collection sort the key/value pairs in ascending order. With the help of a key, we can easily search, or remove elements. The SortedList<TKey, TValue> class belongs to System.Collections.Generic namespace.

Let's see an example of a generic SortedList<TKey, TValue> collection class in C# that stores elements using Add() method and iterates elements using the for-each loop. Here, we are using KeyValuePair class to get keys and values.

```
using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericSortedListDemo
    {
        public static void Main(string[] args)
        {
            SortedList<int, string> sortedList = new SortedList<int, string>();
            sortedList.Add(1, "One");
            sortedList.Add(5, "Five");
            sortedList.Add(2, "Two");
            sortedList.Add(4, "Four");
            sortedList.Add(3, "Three");

            Console.WriteLine("SortedList Elements: ");
            foreach (KeyValuePair<int, string> kvp in sortedList)
            {
                Console.WriteLine($"Key: {kvp.Key}, Value: {kvp.Value}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
SortedList Elements:
Key: 1, Value: One
Key: 2, Value: Two
Key: 3, Value: Three
Key: 4, Value: Four
Key: 5, Value: Five
```

For more detailed information on Generic SortedList<TKey, TValue> Collection Class in C#, please click on the below URL.

<https://dotnettutorials.net/lesson/generic-sortedlist-collection-class-in-csharp/>

LinkedList<T> Class in C#:

The Generic LinkedList<T> Collection Class in C# uses the concept of a linked list. It allows us to insert and delete elements fastly. It can have duplicate elements. The LinkedList<T> Collection Class belongs to System.Collections.Generic namespace. It allows us to add and remove elements before or after the last index. Let's see an example of a generic LinkedList<T> collection class in C# that stores elements using AddLast() and AddFirst() methods and iterates elements using the for-each loop.

```
using System;
using System.Collections.Generic;
namespace GenericCollections
{
    public class GenericSortedDictionaryDemo
    {
        public static void Main(string[] args)
        {
            LinkedList<string> linkedList = new LinkedList<string>();
            linkedList.AddLast("One");
            linkedList.AddLast("Two");
            linkedList.AddLast("Three");
            linkedList.AddLast("Four");
            linkedList.AddFirst("Five"); //Added to first index
            Console.WriteLine("SortedDictionary Elements: ");
            foreach (var item in linkedList)
            {
                Console.WriteLine($"{item} ");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
SortedDictionary Elements:  
Five  
One  
Two  
Three  
Four
```

For more detailed information on Generic LinkedList<T> Collection Class in C#, please click on the below URL.

<https://dotnettutorials.net/lesson/generic-linkedlist-collection-class-in-csharp/>

Before understanding Generic Collection Classes in C# and how they work, first, we need to understand one concept called Generics. So, in the next article, I am going to discuss [How to Implement Generics in C#](#) with Examples. Here, in this article, I gave a brief introduction to **Generic Collections in C#**. I hope this article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.